

HIERARCHICAL & GRAPH PATTERNS

Senior Developer Mastery Guide

5 Storage Models · 12 Patterns · Graph Algorithms · 35 Graded Exercises · Full Answer Key

Trees, DAGs, Networks — every SQL technique used in production

TABLE OF CONTENTS

Chapter 1 Graph Theory Primer for SQL Developers
Chapter 2 Storage Model 1 — Adjacency List
Chapter 3 Storage Model 2 — Nested Sets (Modified Preorder Tree Traversal)
Chapter 4 Storage Model 3 — Closure Table
Chapter 5 Storage Model 4 — Materialised Path
Chapter 6 Storage Model 5 — Nested Intervals & Rational Numbers
Chapter 7 Model Comparison & Selection Guide
Chapter 8 Pattern 1 — Top-Down & Bottom-Up Tree Traversal
Chapter 9 Pattern 2 — Subtree Operations (Insert, Move, Delete)
Chapter 10 Pattern 3 — Lowest Common Ancestor (LCA)
Chapter 11 Pattern 4 — Breadth-First Search (BFS) in SQL
Chapter 12 Pattern 5 — Depth-First Search (DFS) & Topological Sort
Chapter 13 Pattern 6 — Shortest Path — Dijkstra in Pure SQL
Chapter 14 Pattern 7 — Transitive Closure & Reachability
Chapter 15 Pattern 8 — Cycle Detection & Graph Validation
Chapter 16 Pattern 9 — Strongly Connected Components (Kosaraju in SQL)
Chapter 17 Pattern 10 — Social Graph Patterns (Mutual Follows, FOAF, Influence)
Chapter 18 Pattern 11 — Bill of Materials & Multi-Level Costing
Chapter 19 Pattern 12 — Access Control & Permission Inheritance
Chapter 20 Performance, Indexing & Anti-Patterns
Chapter 21 35 Graded Exercises
Chapter 22 Full Answer Key

Chapter 1 · Graph Theory Primer for SQL Developers

Core Vocabulary Every SQL Developer Must Know

- Node / Vertex** — An entity in the graph (employee, city, product, page).
- Edge / Arc** — A relationship between two nodes (reports-to, road, depends-on).
- Directed Graph** — Edges have direction: $A \rightarrow B$ does NOT imply $B \rightarrow A$. Org charts, dependencies.
- Undirected Graph** — Edges are bidirectional. Social friendships, road networks.
- Weighted Graph** — Edges carry a numeric value (distance, cost, capacity).
- Tree** — A connected, acyclic directed graph with exactly one root and one path between any two nodes.
- DAG** — Directed Acyclic Graph — directed, no cycles. Package dependencies, workflows.
- Cycle** — A path that starts and ends at the same node. Kills naive recursion.
- Root** — A node with no incoming edges (no parent). Top of a tree.
- Leaf** — A node with no outgoing edges (no children).
- Depth** — Distance (in hops) from the root to a given node.
- Height** — Maximum depth of any leaf in the subtree rooted at a node.
- Ancestor** — Any node on the path from a node up to the root.
- Descendant** — Any node reachable by following edges downward from a node.
- Sibling** — Nodes sharing the same parent.
- LCA** — Lowest Common Ancestor — the deepest node that is an ancestor of both A and B.
- BFS** — Breadth-First Search — explore level by level. Finds shortest hop-count paths.
- DFS** — Depth-First Search — explore as deep as possible before backtracking.
- Topological Sort** — Ordering of nodes such that every edge $A \rightarrow B$ has A before B. Valid only on DAGs.
- SCC** — Strongly Connected Component — maximal subgraph where every node reaches every other.

Graph Types and Their SQL Representations

Graph Type	Examples	SQL Storage Challenge	Key Operation
Rooted Tree	Org chart, file system	Adjacency list / nested sets	Subtree query
Forest	Multiple org trees	Multiple roots in adj. list	Root detection
DAG	Dependencies, workflows	Closure table essential	Topological sort
Undirected Graph	Social network, roads	Store both (A,B) and (B,A)	BFS / shortest path
Directed Graph	Web links, citations	Single edge direction	Reachability
Weighted Graph	Roads with distances	Edge table + weight column	Dijkstra / MST

CONCEPT: Why Trees Are Hard in SQL

SQL is built on set theory — it excels at flat, rectangular data. Trees and graphs are recursive structures. There is no native 'follow all edges' operator in standard SQL. Every technique in this guide is a workaround that trades query complexity for query performance, or storage complexity for query simplicity. Choosing the right model IS the architecture decision.

Chapter 2 · Storage Model 1 — Adjacency List

Structure

Each row stores a node and a direct reference to its parent. The simplest model — mirrors the real-world relationship directly.

◆ Adjacency List schema + sample data

```
CREATE TABLE category (  
  
  cat_id INT PRIMARY KEY,  
  
  name VARCHAR(100),  
  
  parent_id INT REFERENCES category(cat_id), -- NULL = root  
  
  sort_order INT DEFAULT 0  
  
);  
  
  
-- Sample data: Electronics > Computers > Laptops  
  
INSERT INTO category VALUES  
  
(1, 'Electronics', NULL, 1),  
  
(2, 'Computers', 1, 1),  
  
(3, 'Laptops', 2, 1),  
  
(4, 'Desktops', 2, 2),  
  
(5, 'Mobile', 1, 2),  
  
(6, 'Phones', 5, 1),  
  
(7, 'Tablets', 5, 2);
```

Strengths & Weaknesses

Operation	Complexity	Notes
Find direct parent	O(1)	Simple JOIN on parent_id
Find direct children	O(k)	WHERE parent_id = X, index essential
Find all ancestors	O(depth)	Requires recursive CTE
Find all descendants	O(N)	Requires recursive CTE

Insert a node	O(1)	Set parent_id — trivial
Move a subtree	O(1)	Update parent_id of the moved root
Delete a subtree	O(N)	Cascade or recursive delete
Depth of a node	O(depth)	Count hops to root
Count all descendants	O(N)	Requires full traversal

All Core Queries

```
-- 1. Direct children

SELECT * FROM category WHERE parent_id = 1;


-- 2. Direct parent

SELECT p.* FROM category c JOIN category p ON c.parent_id = p.cat_id

WHERE c.cat_id = 3;


-- 3. All ancestors (bottom-up)

WITH RECURSIVE anc AS (

SELECT cat_id, name, parent_id FROM category WHERE cat_id = 3

UNION ALL

SELECT p.cat_id, p.name, p.parent_id FROM category p

JOIN anc ON p.cat_id = anc.parent_id

)

SELECT * FROM anc;


-- 4. All descendants (top-down)

WITH RECURSIVE desc_tree AS (

SELECT cat_id, name, parent_id, 0 AS depth FROM category WHERE cat_id = 1

UNION ALL

SELECT c.cat_id, c.name, c.parent_id, d.depth + 1

FROM category c JOIN desc_tree d ON c.parent_id = d.cat_id

)
```

```
SELECT depth, REPEAT(' ', depth) || name AS indented FROM desc_tree ORDER BY depth;
```

```
-- 5. Siblings
```

```
SELECT s.* FROM category c
```

```
JOIN category s ON s.parent_id = c.parent_id AND s.cat_id <> c.cat_id
```

```
WHERE c.cat_id = 3;
```

```
-- 6. Leaves (no children)
```

```
SELECT c.* FROM category c
```

```
LEFT JOIN category ch ON ch.parent_id = c.cat_id
```

```
WHERE ch.cat_id IS NULL;
```

```
-- 7. Move subtree: make Phones a child of Computers
```

```
UPDATE category SET parent_id = 2 WHERE cat_id = 6;
```

■ *Index parent_id — it is the single most impactful optimisation for adjacency list queries.*

Chapter 3 · Storage Model 2 — Nested Sets (MPTT)

CONCEPT: The Nested Sets Mental Model

Imagine walking the tree twice — a left-to-right DFS. The first time you visit a node, assign lft. The second time (after visiting all children), assign rgt. A node A is an ancestor of B if and only if $A.lft < B.lft$ AND $A.rgt > B.rgt$. An ancestor's rgt is always greater than all its descendants' rgt values.

◆ Nested Sets schema

```
CREATE TABLE nested_category (

  cat_id INT PRIMARY KEY,

  name VARCHAR(100),

  lft INT NOT NULL,

  rgt INT NOT NULL,

  depth INT NOT NULL DEFAULT 0

);

-- Electronics(1,14), Computers(2,9), Laptops(3,4), Desktops(5,6),
-- Cameras(7,8), Mobile(10,13), Phones(11,12), Tablets... omitted

INSERT INTO nested_category VALUES

(1, 'Electronics', 1, 14, 0), (2, 'Computers', 2, 9, 1),

(3, 'Laptops', 3, 4, 2), (4, 'Desktops', 5, 6, 2), (5, 'Cameras', 7, 8, 2),

(6, 'Mobile', 10, 13, 1), (7, 'Phones', 11, 12, 2);
```

The Power Queries — O(1) Without Recursion

```
-- All descendants of Electronics (no recursion!)

SELECT child.* FROM nested_category child

JOIN nested_category parent ON parent.name = 'Electronics'

WHERE child.lft BETWEEN parent.lft AND parent.rgt

AND child.cat_id <> parent.cat_id;
```

```

-- All ancestors of Laptops (no recursion!)

SELECT anc.* FROM nested_category anc

JOIN nested_category node ON node.name = 'Laptops'

WHERE anc.lft < node.lft AND anc.rgt > node.rgt

ORDER BY anc.lft;

-- Direct parent only

SELECT parent.* FROM nested_category parent

JOIN nested_category child ON child.name = 'Laptops'

WHERE parent.lft < child.lft AND parent.rgt > child.rgt

ORDER BY parent.lft DESC LIMIT 1;

-- Leaf nodes (rgt = lft + 1)

SELECT * FROM nested_category WHERE rgt = lft + 1;

-- Count descendants

SELECT name, (rgt - lft - 1) / 2 AS descendant_count

FROM nested_category;

```

Inserting a Node (the painful part)

```

-- Insert new leaf 'Gaming Laptops' under Laptops (lft=3, rgt=4)

BEGIN;

-- Step 1: Make room – shift all rgt values >= 4

UPDATE nested_category SET rgt = rgt + 2 WHERE rgt >= 4;

-- Step 2: Shift all lft values >= 4

UPDATE nested_category SET lft = lft + 2 WHERE lft >= 4;

-- Step 3: Insert the new node

INSERT INTO nested_category VALUES (8, 'Gaming Laptops', 4, 5, 3);

COMMIT;

```

■ Every insert or move in nested sets requires updating $O(N)$ rows. NEVER use nested sets for write-heavy trees. Use adjacency list + recursive CTE instead.

Chapter 4 · Storage Model 3 — Closure Table

CONCEPT: The Closure Table Idea

Store every (ancestor, descendant, depth) pair explicitly — including self-referential pairs where ancestor = descendant (depth=0). This precomputes the entire transitive closure. Any ancestor/descendant query becomes a simple lookup with no recursion, at the cost of $O(N^2)$ storage in the worst case (very deep, wide trees).

◆ Closure Table schema

```
CREATE TABLE category (cat_id INT PRIMARY KEY, name VARCHAR(100));

CREATE TABLE category_closure (

  ancestor INT REFERENCES category(cat_id),

  descendant INT REFERENCES category(cat_id),

  depth INT NOT NULL,

  PRIMARY KEY (ancestor, descendant)

);

-- For the tree: Electronics(1) > Computers(2) > Laptops(3)

-- Closure table rows:

INSERT INTO category_closure VALUES

(1,1,0),(1,2,1),(1,3,2), -- Electronics ancestors itself, Computers, Laptops

(2,2,0),(2,3,1), -- Computers ancestors itself and Laptops

(3,3,0); -- Laptops ancestors only itself
```

All Core Queries — Elegant and Fast

```
-- All descendants of Electronics

SELECT c.* FROM category c

JOIN category_closure cl ON cl.descendant = c.cat_id

WHERE cl.ancestor = 1 AND cl.depth > 0;

-- All ancestors of Laptops
```

```

SELECT c.* FROM category c

JOIN category_closure cl ON cl.ancestor = c.cat_id

WHERE cl.descendant = 3 AND cl.depth > 0

ORDER BY cl.depth DESC;

-- Direct children only (depth = 1)

SELECT c.* FROM category c

JOIN category_closure cl ON cl.descendant = c.cat_id

WHERE cl.ancestor = 1 AND cl.depth = 1;

-- Direct parent only

SELECT c.* FROM category c

JOIN category_closure cl ON cl.ancestor = c.cat_id

WHERE cl.descendant = 3 AND cl.depth = 1;

-- Depth of any node

SELECT depth FROM category_closure WHERE ancestor = 1 AND descendant = 3;

```

Insert, Move, Delete Operations

```

-- INSERT a new leaf node (new_id = 4, parent = 3 'Laptops')

INSERT INTO category VALUES (4, 'Gaming Laptops');

INSERT INTO category_closure (ancestor, descendant, depth)

SELECT cl.ancestor, 4, cl.depth + 1

FROM category_closure cl

WHERE cl.descendant = 3 -- parent's ancestors

UNION ALL

SELECT 4, 4, 0; -- self-reference

-- DELETE a leaf node (and its closure rows)

```

```

DELETE FROM category_closure WHERE descendant = 4;

DELETE FROM category WHERE cat_id = 4;

-- MOVE subtree: move node 3 (Laptops) under node 6 (Mobile)

-- Step 1: Delete connections between old ancestors and the subtree

DELETE FROM category_closure

WHERE descendant IN (SELECT descendant FROM category_closure WHERE ancestor = 3)

AND ancestor NOT IN (SELECT descendant FROM category_closure WHERE ancestor = 3);

-- Step 2: Re-insert with new parent chain

INSERT INTO category_closure (ancestor, descendant, depth)

SELECT supertree.ancestor,

subtree.descendant,

supertree.depth + subtree.depth + 1

FROM category_closure AS supertree

CROSS JOIN category_closure AS subtree

WHERE supertree.descendant = 6 -- new parent

AND subtree.ancestor = 3; -- subtree root

```

■ Closure table is the BEST model for read-heavy hierarchies with complex ancestor/descendant queries. Inserts are slightly expensive but writes are $O(\text{depth})$, not $O(N)$.

Chapter 5 · Storage Model 4 — Materialised Path

◆ Materialised Path schema

```
CREATE TABLE category (

cat_id INT PRIMARY KEY,

name VARCHAR(100),

path VARCHAR(500) NOT NULL, -- e.g. '/1/2/3/'

depth INT GENERATED ALWAYS AS (LENGTH(path) - LENGTH(REPLACE(path, '/', ''))) - 1 STORED

);


INSERT INTO category VALUES

(1, 'Electronics', '/1/', DEFAULT),

(2, 'Computers', '/1/2/', DEFAULT),

(3, 'Laptops', '/1/2/3/', DEFAULT),

(4, 'Mobile', '/1/4/', DEFAULT),

(5, 'Phones', '/1/4/5/', DEFAULT);
```

Queries Using Path LIKE

```
-- All descendants of Electronics (/1/)

SELECT * FROM category WHERE path LIKE '/1/%' AND cat_id <> 1;


-- All ancestors of Laptops (/1/2/3/)

SELECT * FROM category

WHERE '/1/2/3/' LIKE path || '%' -- our path starts with ancestor path

AND cat_id <> 3

ORDER BY depth;


-- Direct children of Computers (/1/2/)
```

```
SELECT * FROM category

WHERE path LIKE '/1/2/%'

AND depth = 3 -- depth of /1/2/X/ = 3

AND cat_id <> 2;

-- Siblings of Laptops

SELECT * FROM category

WHERE path LIKE '/1/2/%' AND depth = 3 AND cat_id <> 3;

-- Move subtree: update path prefix for all nodes in subtree

UPDATE category

SET path = '/1/4/' || SUBSTRING(path FROM LENGTH('/1/2/') + 1)

WHERE path LIKE '/1/2/%'; -- moves Computers subtree under Mobile
```

Operation	Materialised Path Performance	Notes
Find descendants	O(N) — LIKE scan	Prefix index helps a lot
Find ancestors	O(N) — reverse LIKE	Slower; closure table better here
Insert leaf	O(1)	Just concatenate parent path + new_id
Move subtree	O(subtree size)	Update all paths with string replace
Sort tree order	O(1)	ORDER BY path gives correct DFS order

■ Create a prefix index (e.g. `varchar_pattern_ops` in PostgreSQL) on the path column. LIKE `'/1/2/%'` queries will use the index; LIKE `'%/2/%'` will not.

Chapter 6 · Storage Model 5 — Nested Intervals & Rational Numbers

An advanced model that encodes each node as a rational number fraction (numer/denom). Children are placed between the parent and its right neighbour using the Stern-Brocot tree. It avoids the re-numbering problem of nested sets while keeping $O(1)$ ancestor queries.

◆ Nested Intervals — theoretical model

```
CREATE TABLE nested_interval_tree (

  node_id INT PRIMARY KEY,

  name VARCHAR(100),

  numer BIGINT NOT NULL, -- left bound numerator

  denom BIGINT NOT NULL, -- left bound denominator

  r_numer BIGINT NOT NULL, -- right bound numerator

  r_denom BIGINT NOT NULL -- right bound denominator

);

-- A is ancestor of B iff:

-- (A.numer/A.denom < B.numer/B.denom)

-- AND (B.r_numer/B.r_denom < A.r_numer/A.r_denom)

-- Ancestor query (no recursion, no LIKE)

SELECT a.* FROM nested_interval_tree a

JOIN nested_interval_tree b ON b.name = 'Laptops'

WHERE a.numer * b.denom < b.numer * a.denom

AND b.r_numer * a.r_denom < a.r_numer * b.r_denom;
```

■ *Nested Intervals are academically elegant but rarely used in practice due to complexity of computing child positions. Closure Table achieves similar $O(1)$ ancestor queries with far simpler implementation.*

Chapter 7 · Model Comparison & Selection Guide

Criterion	Adjacency List	Nested Sets	Closure Table	Mat. Path
Find children	$O(1)$ ✓✓	$O(k)$ ✓	$O(k)$ ✓	$O(k)$ ✓
Find all descendants	$O(N)$ —	$O(k)$ ✓✓	$O(k)$ ✓✓	$O(k)$ ✓
Find all ancestors	$O(\text{depth})$ —	$O(\text{depth})$ ✓	$O(\text{depth})$ ✓✓	$O(N)$ —
Insert node	$O(1)$ ✓✓	$O(N)$ ✕✕	$O(\text{depth})$ ✓	$O(1)$ ✓✓
Move subtree	$O(1)$ ✓✓	$O(N)$ ✕✕	$O(\text{sub})$ ✓	$O(\text{sub})$ ✓
Delete subtree	$O(N)$ —	$O(N)$ ✕	$O(\text{sub})$ ✓	$O(\text{sub})$ ✓
Storage overhead	Minimal ✓✓	Minimal ✓✓	$O(N^2)$ ✕	Medium ✓
Depth of node	$O(\text{depth})$	$O(1)$ ✓✓	$O(1)$ ✓✓	$O(1)$ ✓✓
Sort tree order	Hard ✕	$O(1)$ ✓✓	Hard ✕	$O(1)$ ✓✓
Portability	All ✓✓	All ✓✓	All ✓✓	All ✓✓

Decision Guide

Read-heavy, few writes, need fast subtree queries?

→ Closure Table (best read performance, moderate storage cost)

Frequent inserts/moves, unlimited depth?

→ Adjacency List + Recursive CTE (simplest writes, moderate read cost)

Need natural sort order, leaf detection by formula?

→ Nested Sets (but accept $O(N)$ write cost – only for static/rarely-changing data)

Need human-readable paths and LIKE-based queries?

→ Materialised Path (good all-rounder, excellent with prefix index)

DAG (multiple parents) rather than a tree?

→ Closure Table ONLY – all other models assume single-parent trees

Need portability + minimal complexity?

→ Adjacency List – it is always a safe default

Chapter 8 · Pattern 1 — Top-Down & Bottom-Up Tree Traversal

8.1 Top-Down with Full Metadata

```
WITH RECURSIVE tree AS (  
  
  SELECT cat_id, name, parent_id,  
  
    0 AS depth,  
  
    CAST(name AS VARCHAR(2000)) AS path_str,  
  
    CAST(cat_id AS VARCHAR(500)) AS id_path,  
  
    cat_id AS sort_key  
  
  FROM category WHERE parent_id IS NULL  
  
  
  UNION ALL  
  
  SELECT c.cat_id, c.name, c.parent_id,  
  
    t.depth + 1,  
  
    t.path_str || ' > ' || c.name,  
  
    t.id_path || '/' || c.cat_id,  
  
    c.sort_order  
  
  FROM category c JOIN tree t ON c.parent_id = t.cat_id  
  
)  
  
SELECT  
  
  depth,  
  
  REPEAT(' ', depth) || name AS indented_name,  
  
  path_str,  
  
  id_path  
  
FROM tree  
  
ORDER BY id_path; -- natural tree order
```

8.2 Bottom-Up: Ancestor Chain to Root

```
WITH RECURSIVE ancestors AS (

  SELECT cat_id, name, parent_id, 0 AS level

  FROM category WHERE cat_id = 3 -- start: Laptops

  UNION ALL

  SELECT p.cat_id, p.name, p.parent_id, a.level + 1

  FROM category p JOIN ancestors a ON p.cat_id = a.parent_id

)

SELECT level, name FROM ancestors ORDER BY level;

-- Result: Laptops(0), Computers(1), Electronics(2)
```

8.3 Breadcrumb Trail for UI

```
-- Returns: 'Electronics > Computers > Laptops'

WITH RECURSIVE crumbs AS (

  SELECT cat_id, name, parent_id,

  CAST(name AS VARCHAR(2000)) AS breadcrumb

  FROM category WHERE cat_id = 3

  UNION ALL

  SELECT p.cat_id, p.name, p.parent_id,

  p.name || ' > ' || c.breadcrumb

  FROM category p JOIN crumbs c ON p.cat_id = c.parent_id

)

SELECT breadcrumb FROM crumbs WHERE parent_id IS NULL;
```

8.4 Count Subtree Size per Node

```
WITH RECURSIVE subtree AS (

  SELECT cat_id AS root_id, cat_id AS node_id

  FROM category
```

```
UNION ALL

SELECT s.root_id, c.cat_id

FROM category c JOIN subtree s ON c.parent_id = s.node_id

)

SELECT root_id, cat.name,

COUNT(node_id) - 1 AS descendant_count -- exclude self

FROM subtree JOIN category cat ON cat.cat_id = root_id

GROUP BY root_id, cat.name

ORDER BY descendant_count DESC;
```

Chapter 9 · Pattern 2 — Subtree Operations (Insert, Move, Delete)

9.1 Safe Recursive Delete (Adjacency List)

```
-- Delete Computers (cat_id=2) and all its descendants

WITH RECURSIVE to_delete AS (

  SELECT cat_id FROM category WHERE cat_id = 2

  UNION ALL

  SELECT c.cat_id FROM category c

  JOIN to_delete td ON c.parent_id = td.cat_id

)

DELETE FROM category WHERE cat_id IN (SELECT cat_id FROM to_delete);
```

9.2 Move Subtree (Adjacency List — trivial)

```
-- Move Computers (cat_id=2) to become child of Mobile (cat_id=5)

UPDATE category SET parent_id = 5 WHERE cat_id = 2;

-- That's it – all descendants automatically follow
```

9.3 Insert with Auto-Ordering

```
-- Insert 'Gaming Laptops' as last child of Laptops (cat_id=3)

INSERT INTO category (cat_id, name, parent_id, sort_order)

SELECT 10, 'Gaming Laptops', 3,

COALESCE(MAX(sort_order), 0) + 1

FROM category WHERE parent_id = 3;
```

9.4 Validate Tree Integrity

```
-- Check 1: Orphan nodes (parent_id references non-existent node)

SELECT c.* FROM category c

LEFT JOIN category p ON c.parent_id = p.cat_id

WHERE c.parent_id IS NOT NULL AND p.cat_id IS NULL;
```

```
-- Check 2: Cycle detection

WITH RECURSIVE cycle_test AS (

  SELECT cat_id, parent_id, ARRAY[cat_id] AS visited, false AS cycle

  FROM category WHERE parent_id IS NULL

  UNION ALL

  SELECT c.cat_id, c.parent_id,

  ct.visited || c.cat_id,

  c.cat_id = ANY(ct.visited)

  FROM category c JOIN cycle_test ct ON c.parent_id = ct.cat_id

  WHERE NOT ct.cycle

)

SELECT cat_id, visited AS cycle_path FROM cycle_test WHERE cycle;

-- Check 3: Multiple roots per tree (if only one root expected)

SELECT COUNT(*) AS root_count FROM category WHERE parent_id IS NULL;
```

Chapter 10 · Pattern 3 — Lowest Common Ancestor (LCA)

CONCEPT: Lowest Common Ancestor

Given two nodes A and B in a tree, their LCA is the deepest node that is an ancestor of both A and B. Use cases: category navigation (find common parent category), org chart (find shared manager), file systems (find shared directory), gene taxonomy (find shared ancestor species).

10.1 LCA via Closure Table (Most Efficient)

```
-- LCA of node 3 (Laptops) and node 7 (Phones)

-- Their common ancestors are those appearing in BOTH ancestor chains

SELECT ca.ancestor AS lca_id, cat.name, MAX(ca.depth) AS depth_from_root

FROM category_closure ca

JOIN category cat ON cat.cat_id = ca.ancestor

WHERE ca.descendant IN (3, 7) -- our two nodes

GROUP BY ca.ancestor, cat.name

HAVING COUNT(DISTINCT ca.descendant) = 2 -- must be ancestor of BOTH

ORDER BY depth_from_root DESC

LIMIT 1; -- deepest = LCA
```

10.2 LCA via Recursive CTE (Adjacency List)

```
-- Get full ancestor chains for both nodes

WITH RECURSIVE

chain_a AS (

SELECT cat_id, parent_id, 0 AS depth FROM category WHERE cat_id = 3

UNION ALL

SELECT p.cat_id, p.parent_id, ca.depth + 1

FROM category p JOIN chain_a ca ON p.cat_id = ca.parent_id

),

chain_b AS (
```

```

SELECT cat_id, parent_id, 0 AS depth FROM category WHERE cat_id = 7

UNION ALL

SELECT p.cat_id, p.parent_id, cb.depth + 1

FROM category p JOIN chain_b cb ON p.cat_id = cb.parent_id

)

-- Intersect ancestor chains, pick deepest common node

SELECT a.cat_id, cat.name FROM chain_a a

JOIN chain_b b ON a.cat_id = b.cat_id

JOIN category cat ON cat.cat_id = a.cat_id

ORDER BY a.depth LIMIT 1;

```

10.3 LCA via Materialised Path (Simplest)

```

-- LCA is the node whose path is the longest common prefix of both paths

WITH paths AS (

SELECT path FROM category WHERE cat_id IN (3, 7)

),

common_prefix AS (

SELECT

SUBSTRING(p1.path FOR

LENGTH(p1.path) - LENGTH(REPLACE(

SUBSTRING(p1.path FOR

(SELECT MIN(i) FROM generate_series(1,LENGTH(p1.path)) i

WHERE SUBSTRING(p1.path,1,i) <> SUBSTRING(p2.path,1,i))

), '/', '')) * 2

) AS lca_path

FROM paths p1 CROSS JOIN paths p2 WHERE p1.path < p2.path

)

```

```
SELECT cat.* FROM category cat  
  
JOIN common_prefix cp ON cat.path = cp.lca_path;
```

■ *Closure Table LCA is $O(\text{depth})$ — the fastest approach. Always prefer it when you have a closure table already built.*

Chapter 11 · Pattern 4 — Breadth-First Search (BFS) in SQL

CONCEPT: BFS in SQL

BFS explores nodes level by level. In SQL, each iteration of a recursive CTE naturally corresponds to one BFS level when using UNION (which deduplicates, preventing revisiting). BFS gives the shortest hop-count path.

Graph Schema — social_network

```
CREATE TABLE follows (

  follower_id INT,

  followee_id INT,

  PRIMARY KEY (follower_id, followee_id)

);

-- Nodes: 1=Alice, 2=Bob, 3=Carol, 4=Dave, 5=Eve

INSERT INTO follows VALUES

(1,2),(1,3),(2,4),(3,4),(4,5),(2,5);
```

11.1 BFS — All Nodes Reachable from Alice

```
WITH RECURSIVE bfs AS (

  -- Level 0: start node

  SELECT followee_id AS node_id, 1 AS hops

  FROM follows WHERE follower_id = 1 -- Alice

  UNION -- deduplication = no revisiting

  SELECT f.followee_id, bfs.hops + 1

  FROM follows f JOIN bfs ON f.follower_id = bfs.node_id

  WHERE bfs.hops < 6 -- depth cap

)
```

```

SELECT node_id, MIN(hops) AS min_hops

FROM bfs

GROUP BY node_id

ORDER BY min_hops;

```

11.2 BFS with Path Tracking

```

WITH RECURSIVE bfs AS (

  SELECT followee_id AS node_id,

  1 AS hops,

  CAST('1->' || followee_id AS VARCHAR(2000)) AS path

  FROM follows WHERE follower_id = 1

  UNION

  SELECT f.followee_id,

  bfs.hops + 1,

  bfs.path || '->' || f.followee_id

  FROM follows f JOIN bfs ON f.follower_id = bfs.node_id

  WHERE bfs.hops < 6

  AND bfs.path NOT LIKE '%' || f.followee_id || '%' -- cycle guard

)

SELECT node_id, MIN(hops) AS hops, MIN(path) AS shortest_path

FROM bfs GROUP BY node_id ORDER BY hops;

```

11.3 Is Node B Reachable from Node A? (Existence Check)

```

WITH RECURSIVE reach AS (

  SELECT followee_id AS node_id FROM follows WHERE follower_id = 1

  UNION

  SELECT f.followee_id FROM follows f

```

```
JOIN reach ON f.follower_id = reach.node_id

)

SELECT EXISTS (SELECT 1 FROM reach WHERE node_id = 5) AS can_reach_eve;
```

Chapter 12 · Pattern 5 — DFS & Topological Sort

12.1 DFS Order Using Materialised Path Sort

```
-- DFS order: ORDER BY path gives correct DFS pre-order traversal

WITH RECURSIVE dfs AS (

  SELECT cat_id, name, parent_id, 0 AS depth,

  CAST(LPAD(CAST(sort_order AS VARCHAR), 5, '0') AS VARCHAR(2000)) AS sort_path

  FROM category WHERE parent_id IS NULL

  UNION ALL

  SELECT c.cat_id, c.name, c.parent_id, d.depth + 1,

  d.sort_path || '.' || LPAD(CAST(c.sort_order AS VARCHAR), 5, '0')

  FROM category c JOIN dfs d ON c.parent_id = d.cat_id

)

SELECT depth, REPEAT(' ', depth) || name AS name_indented

FROM dfs ORDER BY sort_path;
```

12.2 Topological Sort (Kahn's Algorithm in SQL)

```
-- tasks: (task_id, task_name), task_deps: (task_id, depends_on)

WITH RECURSIVE

in_degree AS (

  SELECT t.task_id, COUNT(d.depends_on) AS indegree

  FROM tasks t

  LEFT JOIN task_deps d ON d.task_id = t.task_id

  GROUP BY t.task_id

),

topo AS (

  -- Start with zero-in-degree nodes
```

```

SELECT task_id, task_name, 0 AS topo_order

FROM tasks WHERE task_id IN (SELECT task_id FROM in_degree WHERE indegree = 0)

UNION ALL

-- Add tasks whose dependencies are all in the completed set

SELECT t.task_id, t.task_name, tp.topo_order + 1

FROM tasks t

JOIN task_deps td ON td.task_id = t.task_id

JOIN topo tp ON tp.task_id = td.depends_on

WHERE NOT EXISTS (

SELECT 1 FROM task_deps td2

WHERE td2.task_id = t.task_id

AND td2.depends_on NOT IN (SELECT task_id FROM topo)

)

)

SELECT DISTINCT task_id, task_name, MIN(topo_order) AS exec_order

FROM topo GROUP BY task_id, task_name ORDER BY exec_order;

```

■ True Kahn's algorithm requires iterative removal of zero-in-degree nodes. This SQL approximation works correctly on DAGs but may produce sub-optimal ordering for tasks with equal precedence. For exact Kahn's, use a stored procedure.

Chapter 13 · Pattern 6 — Shortest Path — Dijkstra in Pure SQL

Graph Schema — `weighted_roads`

```
CREATE TABLE roads (

  from_city VARCHAR(50), to_city VARCHAR(50), km INT

);

-- Bidirectional graph: insert both directions

INSERT INTO roads VALUES

('Dubai','Abu Dhabi',150),('Abu Dhabi','Dubai',150),

('Dubai','Sharjah',30), ('Sharjah','Dubai',30),

('Sharjah','Fujairah',120),('Fujairah','Sharjah',120),

('Abu Dhabi','Al Ain',160),('Al Ain','Abu Dhabi',160),

('Al Ain','Fujairah',130), ('Fujairah','Al Ain',130),

('Dubai','Al Ain',150), ('Al Ain','Dubai',150);
```

13.1 Dijkstra — Shortest Distance to All Cities

```
WITH RECURSIVE dijkstra AS (

  -- Anchor: source city with distance 0

  SELECT CAST('Dubai' AS VARCHAR(50)) AS city,

  0 AS total_km,

  CAST('Dubai' AS VARCHAR(2000)) AS route

  UNION ALL

  -- Expand to unvisited neighbours

  SELECT r.to_city,

  d.total_km + r.km,

  d.route || ' -> ' || r.to_city
```

```

FROM roads r

JOIN dijkstra d ON r.from_city = d.city

WHERE d.route NOT LIKE '%' || r.to_city || '%' -- not-yet-visited

AND d.total_km + r.km < 1000 -- distance cap

)

SELECT city,

MIN(total_km) AS shortest_km,

(SELECT route FROM dijkstra d2

WHERE d2.city = dijkstra.city

ORDER BY total_km LIMIT 1) AS best_route

FROM dijkstra

GROUP BY city

ORDER BY shortest_km;

```

13.2 Single Pair Shortest Path with Full Route

```

-- Shortest path from Dubai to Fujairah

WITH RECURSIVE sp AS (

SELECT CAST('Dubai' AS VARCHAR(50)) AS city, 0 AS km,

CAST('Dubai' AS VARCHAR(2000)) AS route

UNION ALL

SELECT r.to_city, sp.km + r.km, sp.route || ' -> ' || r.to_city

FROM roads r JOIN sp ON r.from_city = sp.city

WHERE sp.route NOT LIKE '%' || r.to_city || '%'

AND sp.city <> 'Fujairah' -- stop expanding once reached

)

SELECT route, km AS total_km FROM sp

WHERE city = 'Fujairah'

```

```
ORDER BY km LIMIT 1;
```

13.3 All Shortest Paths (Multiple Optimal Routes)

```
WITH RECURSIVE all_paths AS (

  SELECT CAST('Dubai' AS VARCHAR(50)) AS city, 0 AS km,

  CAST('Dubai' AS VARCHAR(2000)) AS route

  UNION ALL

  SELECT r.to_city, ap.km + r.km, ap.route || ' -> ' || r.to_city

  FROM roads r JOIN all_paths ap ON r.from_city = ap.city

  WHERE ap.route NOT LIKE '%' || r.to_city || '%'

),

min_dist AS (

  SELECT city, MIN(km) AS min_km FROM all_paths GROUP BY city

)

SELECT ap.city, ap.km, ap.route

FROM all_paths ap JOIN min_dist md ON ap.city = md.city AND ap.km = md.min_km

ORDER BY ap.city, ap.route;
```

■ True Dijkstra requires a priority queue. SQL CTEs implement BFS-style Dijkstra which is correct for small graphs. For large weighted graphs (>10k nodes), use a dedicated graph database or pgRouting (PostgreSQL extension).

Chapter 14 · Pattern 7 — Transitive Closure & Reachability

14.1 Full Transitive Closure with Hop Count

```
WITH RECURSIVE closure AS (

  SELECT follower_id AS src, followee_id AS dst, 1 AS hops

  FROM follows

  UNION

  SELECT c.src, f.followee_id, c.hops + 1

  FROM closure c JOIN follows f ON f.follower_id = c.dst

  WHERE c.hops < 10

)

SELECT src, dst, MIN(hops) AS min_hops

FROM closure

GROUP BY src, dst

ORDER BY src, min_hops;
```

14.2 Reachability Matrix

```
-- Boolean reachability: can src reach dst?

WITH RECURSIVE closure AS (

  SELECT follower_id AS src, followee_id AS dst FROM follows

  UNION

  SELECT c.src, f.followee_id

  FROM closure c JOIN follows f ON f.follower_id = c.dst

)

SELECT

  n1.node_id AS src,

  n2.node_id AS dst,
```

```

EXISTS (

SELECT 1 FROM closure c

WHERE c.src = n1.node_id AND c.dst = n2.node_id

) AS can_reach

FROM (SELECT DISTINCT follower_id AS node_id FROM follows) n1

CROSS JOIN (SELECT DISTINCT follower_id AS node_id FROM follows) n2;

```

14.3 Influence Score (Weighted Reachability)

```

-- Influence decays by 50% per hop: score = SUM(1 / 2^hops)

WITH RECURSIVE closure AS (

SELECT follower_id AS src, followee_id AS dst, 1 AS hops

FROM follows

UNION

SELECT c.src, f.followee_id, c.hops + 1

FROM closure c JOIN follows f ON f.follower_id = c.dst

WHERE c.hops < 6

)

SELECT src,

SUM(1.0 / POWER(2, hops)) AS influence_score,

COUNT(DISTINCT dst) AS reach_count

FROM (SELECT src, dst, MIN(hops) AS hops FROM closure GROUP BY src, dst) ranked

GROUP BY src

ORDER BY influence_score DESC;

```

Chapter 15 · Pattern 8 — Cycle Detection & Graph Validation

15.1 Detect Cycles in Directed Graph (Array Method)

```
WITH RECURSIVE walk AS (

  SELECT follower_id AS start, followee_id AS current,

  ARRAY[follower_id, followee_id] AS visited,

  false AS has_cycle

  FROM follows

  UNION ALL

  SELECT w.start, f.followee_id,

  w.visited || f.followee_id,

  f.followee_id = ANY(w.visited)

  FROM follows f JOIN walk w ON f.follower_id = w.current

  WHERE NOT w.has_cycle AND array_length(w.visited, 1) < 20

)

SELECT DISTINCT start AS cycle_entry, visited AS cycle_path

FROM walk WHERE has_cycle;
```

15.2 Check if Graph is a Valid DAG

```
-- A graph is a DAG iff no cycle exists

WITH RECURSIVE walk AS (

  SELECT follower_id AS start, followee_id AS dst,

  CAST(follower_id AS VARCHAR(500)) AS path

  FROM follows

  UNION ALL

  SELECT w.start, f.followee_id, w.path || ',' || f.followee_id

  FROM follows f JOIN walk w ON f.follower_id = w.dst
```

```

WHERE w.path NOT LIKE '%, ' || CAST(f.followee_id AS VARCHAR) || '%'

)

SELECT

CASE WHEN EXISTS (

SELECT 1 FROM walk WHERE start = dst

) THEN 'CYCLIC — NOT a DAG'

ELSE 'ACYCLIC — Valid DAG'

END AS graph_type;

```

15.3 Find All Nodes Involved in Any Cycle

```

WITH RECURSIVE walk AS (

SELECT follower_id AS origin, followee_id AS current,

ARRAY[follower_id] AS visited

FROM follows

UNION ALL

SELECT w.origin, f.followee_id, w.visited || f.followee_id

FROM follows f JOIN walk w ON f.follower_id = w.current

WHERE NOT (f.followee_id = ANY(w.visited))

AND array_length(w.visited, 1) < 15

)

SELECT DISTINCT origin AS node_in_cycle

FROM walk WHERE current = origin;

```

■ *Cycle detection queries are expensive — $O(V \cdot E)$ in the worst case. Run them as data-quality checks, not in hot query paths. Add a depth cap (`array_length < N`) unconditionally.*

Chapter 16 · Pattern 9 — Strongly Connected Components

CONCEPT: Strongly Connected Components (SCCs)

In a directed graph, an SCC is a maximal set of nodes where every node can reach every other node. In an org chart, SCCs > 1 indicate illegal cycles. In social networks, SCCs reveal tight communities. In dependency graphs, SCCs reveal circular dependencies that must be broken.

16.1 SCC Identification via Mutual Reachability

```
-- Two nodes are in the same SCC if A can reach B AND B can reach A

WITH RECURSIVE fwd AS (

  SELECT follower_id AS src, followee_id AS dst FROM follows

  UNION

  SELECT f.src, follows.followee_id FROM fwd f

  JOIN follows ON follows.follower_id = f.dst

),

bwd AS (

  SELECT followee_id AS src, follower_id AS dst FROM follows

  UNION

  SELECT b.src, follows.follower_id FROM bwd b

  JOIN follows ON follows.followee_id = b.dst

)

-- Nodes where mutual reachability holds

SELECT DISTINCT fwd.src AS node_a, fwd.dst AS node_b

FROM fwd JOIN bwd ON fwd.src = bwd.dst AND fwd.dst = bwd.src

WHERE fwd.src < fwd.dst -- unique pairs

ORDER BY node_a;
```

16.2 Label Each Node with Its SCC ID

```
-- Assign minimum reachable node ID as SCC label
```

```
WITH RECURSIVE

forward AS (

SELECT follower_id AS src, followee_id AS dst FROM follows

UNION

SELECT f.src, nxt.followee_id FROM forward f JOIN follows nxt ON nxt.follower_id = f.dst

),

backward AS (

SELECT followee_id AS src, follower_id AS dst FROM follows

UNION

SELECT b.src, prv.follower_id FROM backward b JOIN follows prv ON prv.followee_id = b.dst

),

scc_pairs AS (

SELECT f.src, f.dst FROM forward f JOIN backward b ON f.src=b.dst AND f.dst=b.src

)

SELECT src AS node_id, MIN(dst) AS scc_label

FROM scc_pairs GROUP BY src

ORDER BY scc_label, node_id;
```

Chapter 17 · Pattern 10 — Social Graph Patterns

17.1 Mutual Followers (Bidirectional Edges)

```
SELECT a.follower_id AS user1, a.followee_id AS user2

FROM follows a JOIN follows b

ON a.follower_id = b.followee_id AND a.followee_id = b.follower_id

AND a.follower_id < a.followee_id; -- unique pairs only
```

17.2 Friend-of-Friend (FOAF) Suggestions

```
-- People Alice follows, but does not yet follow directly

SELECT DISTINCT foaf.followee_id AS suggested_user

FROM follows direct

JOIN follows foaf ON foaf.follower_id = direct.followee_id

WHERE direct.follower_id = 1 -- Alice

AND foaf.followee_id <> 1 -- not Alice herself

AND NOT EXISTS ( -- Alice doesn't already follow them

SELECT 1 FROM follows existing

WHERE existing.follower_id = 1

AND existing.followee_id = foaf.followee_id

);
```

17.3 Degree of Separation

```
WITH RECURSIVE degrees AS (

SELECT followee_id AS dst, 1 AS deg FROM follows WHERE follower_id = 1

UNION

SELECT f.followee_id, d.deg + 1

FROM follows f JOIN degrees d ON f.follower_id = d.dst

WHERE d.deg < 6
```

```

)

SELECT dst AS user_id, MIN(deg) AS degree_of_separation

FROM degrees WHERE dst <> 1

GROUP BY dst ORDER BY degree_of_separation;

```

17.4 Most Influential Node (PageRank Approximation)

```

-- Simplified PageRank: count weighted in-links

WITH RECURSIVE closure AS (

SELECT follower_id AS src, followee_id AS dst, 1 AS hops

FROM follows

UNION

SELECT c.src, f.followee_id, c.hops + 1

FROM closure c JOIN follows f ON f.follower_id = c.dst

WHERE c.hops < 4

)

SELECT dst AS user_id,

COUNT(DISTINCT src) AS unique_reachers,

SUM(1.0 / POWER(2, hops)) AS weighted_influence

FROM closure

GROUP BY dst

ORDER BY weighted_influence DESC;

```

Chapter 18 · Pattern 11 — Bill of Materials & Multi-Level Costing

◆ Bill of Materials schema

```
CREATE TABLE parts (

  part_id INT PRIMARY KEY,

  name VARCHAR(100),

  unit_cost DECIMAL(10,2)

);


CREATE TABLE bom ( -- Bill of Materials

  parent_part INT REFERENCES parts(part_id),

  child_part INT REFERENCES parts(part_id),

  quantity INT NOT NULL

);


-- Car(1) needs Engine(2) x1, Wheels(3) x4

-- Engine(2) needs Block(4) x1, Pistons(5) x8
```

18.1 Explode BOM — All Components of a Product

```
WITH RECURSIVE bom_explosion AS (

  SELECT child_part AS part_id, quantity, 1 AS depth,

  quantity AS total_qty

  FROM bom WHERE parent_part = 1 -- start: Car

  UNION ALL

  SELECT b.child_part,

  b.quantity,

  be.depth + 1,
```

```

be.total_qty * b.quantity -- multiply quantities up the chain

FROM bom b JOIN bom_explosion be ON b.parent_part = be.part_id

)

SELECT p.name, SUM(be.total_qty) AS total_needed,

p.unit_cost,

SUM(be.total_qty) * p.unit_cost AS total_cost

FROM bom_explosion be JOIN parts p ON p.part_id = be.part_id

GROUP BY p.part_id, p.name, p.unit_cost

ORDER BY total_cost DESC;

```

18.2 Where-Used: Find All Products Using a Part

```

-- Which top-level products use Pistons (part_id=5)?

WITH RECURSIVE where_used AS (

SELECT parent_part AS part_id FROM bom WHERE child_part = 5

UNION ALL

SELECT b.parent_part FROM bom b JOIN where_used wu ON b.child_part = wu.part_id

)

SELECT DISTINCT p.name FROM where_used wu JOIN parts p ON p.part_id = wu.part_id;

```

Chapter 19 · Pattern 12 — Access Control & Permission Inheritance

◆ RBAC with inheritance schema

```
CREATE TABLE roles (role_id INT PRIMARY KEY, role_name VARCHAR(50));

CREATE TABLE role_hierarchy (parent_role INT, child_role INT);

CREATE TABLE role_permissions (role_id INT, permission VARCHAR(100));

CREATE TABLE user_roles (user_id INT, role_id INT);

-- Admin > Manager > Employee

-- Manager inherits Employee's permissions; Admin inherits all
```

19.1 Effective Permissions for a User (with Inheritance)

```
WITH RECURSIVE role_tree AS (

  -- Start with roles directly assigned to user

  SELECT ur.role_id

  FROM user_roles ur WHERE ur.user_id = 42

  UNION ALL

  -- Climb the role hierarchy to inherit parent permissions

  SELECT rh.parent_role

  FROM role_hierarchy rh JOIN role_tree rt ON rh.child_role = rt.role_id

)

SELECT DISTINCT rp.permission

FROM role_tree rt JOIN role_permissions rp ON rp.role_id = rt.role_id

ORDER BY rp.permission;
```

19.2 Has Permission? (Boolean Check)

```
WITH RECURSIVE role_tree AS (
```

```

SELECT role_id FROM user_roles WHERE user_id = 42

UNION ALL

SELECT rh.parent_role FROM role_hierarchy rh

JOIN role_tree rt ON rh.child_role = rt.role_id

)

SELECT EXISTS (

SELECT 1 FROM role_tree rt

JOIN role_permissions rp ON rp.role_id = rt.role_id

WHERE rp.permission = 'DELETE_INVOICE'

) AS has_permission;

```

19.3 Audit: Which Role Grants a Permission?

```

WITH RECURSIVE role_tree AS (

SELECT role_id, role_id AS origin_role, 0 AS depth

FROM user_roles WHERE user_id = 42

UNION ALL

SELECT rh.parent_role, rt.origin_role, rt.depth + 1

FROM role_hierarchy rh JOIN role_tree rt ON rh.child_role = rt.role_id

)

SELECT r.role_name AS granting_role, rt.depth AS inheritance_hops,

rp.permission

FROM role_tree rt

JOIN role_permissions rp ON rp.role_id = rt.role_id

JOIN roles r ON r.role_id = rt.role_id

WHERE rp.permission = 'DELETE_INVOICE'

ORDER BY rt.depth;

```

Chapter 20 · Performance, Indexing & Anti-Patterns

20.1 Index Checklist by Model

Model	Critical Indexes	Optional Indexes
Adjacency List	parent_id	(parent_id, sort_order), (parent_id, cat_id)
Nested Sets	(lft, rgt)	(lft, rgt) composite, depth
Closure Table	(ancestor), (descendant)	(ancestor, depth), (descendant, depth)
Materialised Path	prefix index on path	depth, (parent_id, sort_order)
Graph edges	(from_node), (to_node)	(from_node, to_node), weight

20.2 Query Complexity by Model

Query	Adj. List	Nested Sets	Closure Table	Mat. Path
Direct children	$O(k)$ ✓	$O(k)$ ✓	$O(k)$ ✓	$O(k)$ ✓
All descendants	$O(N)$ —	$O(k)$ ✓✓	$O(k)$ ✓✓	$O(k)$ ✓
All ancestors	$O(d)$ —	$O(d)$ ✓	$O(d)$ ✓✓	$O(N)$ —
Depth	$O(d)$	$O(1)$ ✓✓	$O(1)$ ✓✓	$O(1)$ ✓✓
LCA	$O(d)$	$O(k)$ ✓	$O(d)$ ✓✓	$O(N)$
Insert leaf	$O(1)$ ✓✓	$O(N)$ ✗	$O(d)$ ✓	$O(1)$ ✓✓
Move subtree	$O(1)$ ✓✓	$O(N)$ ✗	$O(s)$ ✓	$O(s)$ ✓

20.3 Top Anti-Patterns

✗ Adjacency list without index on parent_id

→ Every child lookup becomes a full table scan. One index, massive speedup.

✗ Nested sets for frequently updated trees

→ Every insert/move updates $O(N)$ rows. Switch to adjacency list + recursive CTE.

✗ Recursive CTE on graph without cycle guard

→ Infinite loop. Always add depth < N or visited-path check.

✗ SELECT * in recursive member

→ Carries all columns through every iteration. Bloats working table. Select minimally.

✗ BFS with UNION ALL on cyclic graph

→ Exponential row explosion. UNION deduplicates; UNION ALL does not.

✗ Closure table without (descendant) index

→ Ancestor queries (WHERE descendant = X) become scans. Both directions need indexes.

✗ Materialised path without prefix index

→ LIKE '/1/2/%' cannot use a standard B-tree index in all dialects. Use GIN or varchar_pattern_ops.

✗ LCA via full recursive traversal

→ Use closure table or materialised path LIKE intersection — orders of magnitude faster.

✗ Unlimited-depth recursive CTE

→ SQL Server defaults to max 100. Always specify your intended depth cap explicitly.

Chapter 21 · 35 Graded Exercises

Three tiers. Use the schemas defined throughout the guide. Attempt each tier before checking Chapter 22.

BEGINNER (Exercises 1–12)

Ex 1. Using the adjacency list category table, write a query to return all direct children of 'Electronics'.

Ex 2. Find the full ancestor chain (all parents up to root) for 'Laptops' using a recursive CTE.

Ex 3. Write a recursive CTE to display the entire category tree with indentation showing depth.

Ex 4. Find all leaf nodes in the category tree (nodes with no children).

Ex 5. Find all sibling categories of 'Computers' (same parent, different node).

Ex 6. Using nested sets, find all descendants of 'Electronics' without recursion.

Ex 7. Using the closure table, find all ancestors of 'Laptops' ordered by depth descending.

Ex 8. Using the materialised path table, find all nodes under 'Computers' using LIKE.

Ex 9. Write a query that counts the number of descendants for each node in the adjacency list.

Ex 10. Find all root nodes (nodes with no parent) in the category tree.

Ex 11. Using the follows social graph, find all users reachable from Alice (user_id=1) within 2 hops.

Ex 12. Find all mutual followers: pairs (A, B) where A follows B and B follows A.

INTERMEDIATE (Exercises 13–25)

Ex 13. Write a recursive CTE that builds the breadcrumb path ('Electronics > Computers > Laptops') for every node.

Ex 14. Insert a new node 'Gaming Laptops' as a child of 'Laptops' in the closure table, updating all required closure rows.

Ex 15. Move the 'Computers' subtree to become a child of 'Mobile' in the adjacency list.

Ex 16. Find the Lowest Common Ancestor of 'Laptops' and 'Phones' using the closure table.

Ex 17. Using the roads weighted graph, find the shortest distance from Dubai to every other city.

Ex 18. Write a query to find all nodes in the category tree at exactly depth 2.

Ex 19. Using the social follows graph, produce FOAF (friend-of-friend) suggestions for Alice: people she doesn't follow but is connected to within 2 hops.

Ex 20. Compute the total subtree salary budget for each manager in the employees adjacency list tree.

Ex 21. Write a query to detect orphan nodes in the category adjacency list (parent_id references a non-existent cat_id).

Ex 22. Using the BOM schema, explode the full bill of materials for a Car and compute the total component cost.

Ex 23. Find the 'height' of each node in the category tree — the number of levels below it to the deepest leaf.

Ex 24. Using the closure table, delete the 'Computers' node and its entire subtree correctly.

Ex 25. Write a recursive CTE to compute each node's depth and also how many siblings it has at the same level.

ADVANCED (Exercises 26–35)

Ex 26. Implement a full Transitive Closure on the follows graph and compute the minimum hop count between every pair of reachable users.

Ex 27. Detect all cycles in the follows directed graph. Return each cycle as a path string.

Ex 28. Implement SCC labelling: assign each node its SCC ID (minimum node_id reachable from both directions).

Ex 29. Write a BFS on the roads graph that finds all cities reachable within 300km of Dubai, with their shortest distance and route.

Ex 30. Using the RBAC schema, write a query that returns the effective permissions of user 42 including inherited permissions from the role hierarchy, and names the specific role that grants each permission.

Ex 31. Implement a 'where-used' analysis for the BOM: given a part_id, find every top-level product (root of BOM) that directly or indirectly uses it.

Ex 32. Write a query that validates the entire category tree: check for cycles, orphans, duplicate (parent, sort_order) pairs, and nodes with more than 10 children — return one row per issue found.

Ex 33. Compute an influence score for every user in the social graph: $\text{score} = \text{SUM}(1 / 2^{\text{hop_count}})$ for all users reachable within 4 hops. Rank by influence.

Ex 34. Find the diameter of the category tree: the longest shortest path between any two leaf nodes.

Ex 35. CHALLENGE: Write a single SQL statement (multiple CTEs allowed) that produces a complete graph report on the follows social network: node_id, user_name, in_degree, out_degree, betweenness_approximation (number of shortest paths passing through this node as intermediate), scc_label, and degree_of_separation from user_id=1. Handle disconnected nodes correctly.

Chapter 22 · Full Answer Key

Answers 1–12 (Beginner)

Answer 1

```
SELECT * FROM category WHERE parent_id = 1; -- 1=Electronics
```

Answer 2

```
WITH RECURSIVE anc AS (

  SELECT cat_id, name, parent_id FROM category WHERE name = 'Laptops'

  UNION ALL

  SELECT p.cat_id, p.name, p.parent_id FROM category p

  JOIN anc ON p.cat_id = anc.parent_id

)

SELECT name FROM anc ORDER BY cat_id;
```

Answer 3

```
WITH RECURSIVE tree AS (

  SELECT cat_id, name, parent_id, 0 AS depth FROM category WHERE parent_id IS NULL

  UNION ALL

  SELECT c.cat_id, c.name, c.parent_id, t.depth + 1

  FROM category c JOIN tree t ON c.parent_id = t.cat_id

)

SELECT depth, REPEAT(' ', depth) || name AS display FROM tree ORDER BY cat_id;
```

Answer 4

```
SELECT c.* FROM category c

LEFT JOIN category ch ON ch.parent_id = c.cat_id

WHERE ch.cat_id IS NULL;
```

Answer 5

```
SELECT s.* FROM category c
```

```
JOIN category s ON s.parent_id = c.parent_id AND s.cat_id <> c.cat_id

WHERE c.name = 'Computers';
```

Answer 6

```
SELECT child.* FROM nested_category child

JOIN nested_category parent ON parent.name = 'Electronics'

WHERE child.lft > parent.lft AND child.rgt < parent.rgt;
```

Answer 7

```
SELECT c.* FROM category c

JOIN category_closure cl ON cl.ancestor = c.cat_id

WHERE cl.descendant = 3 AND cl.depth > 0

ORDER BY cl.depth DESC;
```

Answer 8

```
SELECT * FROM category

WHERE path LIKE (SELECT path FROM category WHERE name = 'Computers') || '%'

AND name <> 'Computers';
```

Answer 9

```
WITH RECURSIVE sub AS (

SELECT cat_id AS root_id, cat_id FROM category

UNION ALL

SELECT s.root_id, c.cat_id FROM category c JOIN sub s ON c.parent_id = s.cat_id

)

SELECT root_id, c.name, COUNT(*) - 1 AS descendants

FROM sub JOIN category c ON c.cat_id = root_id

GROUP BY root_id, c.name ORDER BY descendants DESC;
```

Answer 10

```
SELECT * FROM category WHERE parent_id IS NULL;
```

Answer 11

```

WITH RECURSIVE bfs AS (

  SELECT followee_id AS node_id, 1 AS hops FROM follows WHERE follower_id = 1

  UNION

  SELECT f.followee_id, bfs.hops + 1

  FROM follows f JOIN bfs ON f.follower_id = bfs.node_id WHERE bfs.hops < 2

)

SELECT node_id, MIN(hops) AS hops FROM bfs GROUP BY node_id;

```

Answer 12

```

SELECT a.follower_id AS user1, a.followee_id AS user2

FROM follows a JOIN follows b

ON a.follower_id = b.followee_id AND a.followee_id = b.follower_id

AND a.follower_id < a.followee_id;

```

Answers 13–25 (Intermediate)**Answer 13**

```

WITH RECURSIVE crumbs AS (

  SELECT cat_id, name, parent_id, CAST(name AS VARCHAR(2000)) AS breadcrumb

  FROM category WHERE parent_id IS NULL

  UNION ALL

  SELECT c.cat_id, c.name, c.parent_id, cr.breadcrumb || ' > ' || c.name

  FROM category c JOIN crumbs cr ON c.parent_id = cr.cat_id

)

SELECT cat_id, name, breadcrumb FROM crumbs ORDER BY breadcrumb;

```

Answer 14

```

-- Insert 'Gaming Laptops' (id=8) as child of Laptops (id=3)

INSERT INTO category VALUES (8, 'Gaming Laptops');

INSERT INTO category_closure (ancestor, descendant, depth)

```

```
SELECT cl.ancestor, 8, cl.depth + 1

FROM category_closure cl WHERE cl.descendant = 3

UNION ALL SELECT 8, 8, 0;
```

Answer 15

```
-- Adjacency list: single update, all descendants follow automatically

UPDATE category SET parent_id = 5 WHERE cat_id = 2; -- 5=Mobile
```

Answer 16

```
SELECT ca.ancestor AS lca_id, cat.name

FROM category_closure ca JOIN category cat ON cat.cat_id = ca.ancestor

WHERE ca.descendant IN (3, 7)

GROUP BY ca.ancestor, cat.name

HAVING COUNT(DISTINCT ca.descendant) = 2

ORDER BY MAX(ca.depth) DESC LIMIT 1;
```

Answer 17

```
WITH RECURSIVE sp AS (

SELECT to_city AS city, km AS total_km FROM roads WHERE from_city = 'Dubai'

UNION ALL

SELECT r.to_city, sp.total_km + r.km

FROM roads r JOIN sp ON r.from_city = sp.city

WHERE sp.total_km + r.km < 1000

)

SELECT city, MIN(total_km) AS shortest_km FROM sp GROUP BY city ORDER BY shortest_km;
```

Answer 18

```
WITH RECURSIVE tree AS (

SELECT cat_id, name, parent_id, 0 AS depth FROM category WHERE parent_id IS NULL

UNION ALL
```

```

SELECT c.cat_id, c.name, c.parent_id, t.depth + 1

FROM category c JOIN tree t ON c.parent_id = t.cat_id WHERE t.depth < 2

)

SELECT cat_id, name FROM tree WHERE depth = 2;

```

Answer 19

```

SELECT DISTINCT f2.followee_id AS suggested

FROM follows f1 JOIN follows f2 ON f2.follower_id = f1.followee_id

WHERE f1.follower_id = 1

AND f2.followee_id <> 1

AND NOT EXISTS (

SELECT 1 FROM follows x WHERE x.follower_id = 1 AND x.followee_id = f2.followee_id

);

```

Answer 20

```

WITH RECURSIVE sub AS (

SELECT emp_id AS root_id, emp_id, salary FROM employees

UNION ALL

SELECT s.root_id, e.emp_id, e.salary

FROM employees e JOIN sub s ON e.manager_id = s.emp_id

)

SELECT root_id, m.name, SUM(salary) AS budget, COUNT(*) AS headcount

FROM sub JOIN employees m ON m.emp_id = root_id

GROUP BY root_id, m.name ORDER BY budget DESC;

```

Answer 21

```

SELECT c.* FROM category c

LEFT JOIN category p ON c.parent_id = p.cat_id

WHERE c.parent_id IS NOT NULL AND p.cat_id IS NULL;

```

Answer 22

```

WITH RECURSIVE bom_exp AS (

  SELECT child_part AS part_id, quantity, quantity AS total_qty

  FROM bom WHERE parent_part = 1

  UNION ALL

  SELECT b.child_part, b.quantity, be.total_qty * b.quantity

  FROM bom b JOIN bom_exp be ON b.parent_part = be.part_id

)

SELECT p.name, SUM(be.total_qty) AS needed,

p.unit_cost, SUM(be.total_qty) * p.unit_cost AS cost

FROM bom_exp be JOIN parts p ON p.part_id = be.part_id

GROUP BY p.part_id, p.name, p.unit_cost ORDER BY cost DESC;

```

Answer 23

```

WITH RECURSIVE tree AS (

  SELECT cat_id, parent_id, 0 AS depth FROM category WHERE parent_id IS NULL

  UNION ALL

  SELECT c.cat_id, c.parent_id, t.depth + 1

  FROM category c JOIN tree t ON c.parent_id = t.cat_id

),

heights AS (

  SELECT p.cat_id AS node_id, MAX(c.depth - p.depth) AS height

  FROM tree p JOIN tree c ON c.cat_id = p.cat_id OR EXISTS (

    WITH RECURSIVE sub AS (

      SELECT cat_id FROM category WHERE cat_id = p.cat_id

      UNION ALL SELECT ch.cat_id FROM category ch JOIN sub s ON ch.parent_id = s.cat_id

    ) SELECT 1 FROM sub WHERE cat_id = c.cat_id

  )

)

```

```

GROUP BY p.cat_id

)

SELECT t.cat_id, cat.name, t.depth, h.height

FROM tree t JOIN category cat ON cat.cat_id = t.cat_id

JOIN heights h ON h.node_id = t.cat_id ORDER BY t.cat_id;

```

Answer 24

```

-- Closure table delete: remove all closure rows where descendant is in subtree of node 2

DELETE FROM category_closure

WHERE descendant IN (

SELECT descendant FROM category_closure WHERE ancestor = 2

);

DELETE FROM category

WHERE cat_id IN (

SELECT descendant FROM category_closure WHERE ancestor = 2

);

```

Answer 25

```

WITH RECURSIVE tree AS (

SELECT cat_id, name, parent_id, 0 AS depth FROM category WHERE parent_id IS NULL

UNION ALL

SELECT c.cat_id, c.name, c.parent_id, t.depth + 1

FROM category c JOIN tree t ON c.parent_id = t.cat_id

)

SELECT t.cat_id, t.name, t.depth,

(SELECT COUNT(*) - 1 FROM tree t2

WHERE t2.parent_id = t.parent_id OR

(t2.parent_id IS NULL AND t.parent_id IS NULL)) AS sibling_count

```

```
FROM tree ORDER BY t.depth, t.cat_id;
```

Answers 26–35 (Advanced)

Answer 26

```
WITH RECURSIVE closure AS (

  SELECT follower_id AS src, followee_id AS dst, 1 AS hops FROM follows

  UNION

  SELECT c.src, f.followee_id, c.hops + 1

  FROM closure c JOIN follows f ON f.follower_id = c.dst WHERE c.hops < 10

)

SELECT src, dst, MIN(hops) AS min_hops FROM closure

GROUP BY src, dst ORDER BY src, min_hops;
```

Answer 27

```
WITH RECURSIVE walk AS (

  SELECT follower_id AS origin, followee_id AS current,

  CAST(follower_id AS VARCHAR(500)) || ',' ||

  CAST(followee_id AS VARCHAR(500)) AS path_str

  FROM follows

  UNION ALL

  SELECT w.origin, f.followee_id,

  w.path_str || ',' || CAST(f.followee_id AS VARCHAR)

  FROM follows f JOIN walk w ON f.follower_id = w.current

  WHERE w.path_str NOT LIKE '%,' || CAST(f.followee_id AS VARCHAR) || ',%'

  AND w.path_str NOT LIKE CAST(f.followee_id AS VARCHAR) || ',%'

  AND LENGTH(w.path_str) < 200

)

SELECT DISTINCT origin, path_str AS cycle_path
```

```
FROM walk WHERE current = origin;
```

Answer 28

```
WITH RECURSIVE

fwd AS (

SELECT follower_id AS src, followee_id AS dst FROM follows

UNION SELECT f.src, nxt.followee_id FROM fwd f JOIN follows nxt ON nxt.follower_id = f.dst

),

bwd AS (

SELECT followee_id AS src, follower_id AS dst FROM follows

UNION SELECT b.src, prv.follower_id FROM bwd b JOIN follows prv ON prv.followee_id = b.dst

)

SELECT f.src AS node_id, MIN(f.dst) AS scc_label

FROM fwd f JOIN bwd b ON f.src = b.dst AND f.dst = b.src

GROUP BY f.src ORDER BY scc_label, node_id;
```

Answer 29

```
WITH RECURSIVE bfs AS (

SELECT to_city AS city, km AS dist,

CAST('Dubai->' || to_city AS VARCHAR(2000)) AS route

FROM roads WHERE from_city = 'Dubai'

UNION

SELECT r.to_city, bfs.dist + r.km, bfs.route || '->' || r.to_city

FROM roads r JOIN bfs ON r.from_city = bfs.city

WHERE bfs.dist + r.km <= 300

AND bfs.route NOT LIKE '%' || r.to_city || '%'

)

SELECT city, MIN(dist) AS shortest_km,
```

```
(SELECT route FROM bfs b2 WHERE b2.city = bfs.city ORDER BY dist LIMIT 1) AS route

FROM bfs GROUP BY city ORDER BY shortest_km;
```

Answer 30

```
WITH RECURSIVE role_tree AS (

  SELECT ur.role_id, ur.role_id AS origin_role, 0 AS hops

  FROM user_roles ur WHERE ur.user_id = 42

  UNION ALL

  SELECT rh.parent_role, rt.origin_role, rt.hops + 1

  FROM role_hierarchy rh JOIN role_tree rt ON rh.child_role = rt.role_id

)

SELECT DISTINCT rp.permission, r.role_name AS granted_by_role, rt.hops AS inheritance_depth

FROM role_tree rt

JOIN role_permissions rp ON rp.role_id = rt.role_id

JOIN roles r ON r.role_id = rt.role_id

ORDER BY rp.permission, rt.hops;
```

Answer 31

```
WITH RECURSIVE where_used AS (

  SELECT parent_part AS part_id FROM bom WHERE child_part = 5 -- Pistons

  UNION ALL

  SELECT b.parent_part FROM bom b JOIN where_used wu ON b.child_part = wu.part_id

)

SELECT DISTINCT p.part_id, p.name AS top_level_product

FROM where_used wu JOIN parts p ON p.part_id = wu.part_id

WHERE NOT EXISTS (SELECT 1 FROM bom WHERE child_part = wu.part_id);
```

Answer 32

```
-- Multi-issue tree validator
```

```

WITH RECURSIVE

cycle_check AS (

SELECT cat_id, parent_id, ARRAY[cat_id] AS v, false AS cyc

FROM category WHERE parent_id IS NULL

UNION ALL

SELECT c.cat_id, c.parent_id, cc.v || c.cat_id, c.cat_id = ANY(cc.v)

FROM category c JOIN cycle_check cc ON c.parent_id = cc.cat_id WHERE NOT cc.cyc

)

SELECT 'CYCLE' AS issue_type, cat_id, NULL AS detail

FROM cycle_check WHERE cyc

UNION ALL

SELECT 'ORPHAN', c.cat_id, NULL FROM category c

LEFT JOIN category p ON c.parent_id = p.cat_id

WHERE c.parent_id IS NOT NULL AND p.cat_id IS NULL

UNION ALL

SELECT 'DUP_SORT', parent_id, COUNT(*)::VARCHAR FROM category

WHERE parent_id IS NOT NULL

GROUP BY parent_id, sort_order HAVING COUNT(*) > 1

UNION ALL

SELECT 'TOO_MANY_CHILDREN', parent_id, COUNT(*)::VARCHAR FROM category

GROUP BY parent_id HAVING COUNT(*) > 10;

```

Answer 33

```

WITH RECURSIVE closure AS (

SELECT follower_id AS src, followee_id AS dst, 1 AS hops FROM follows

UNION

SELECT c.src, f.followee_id, c.hops + 1

```

```

FROM closure c JOIN follows f ON f.follower_id = c.dst WHERE c.hops < 4

),

best AS (

SELECT src, dst, MIN(hops) AS hops FROM closure GROUP BY src, dst

)

SELECT dst AS user_id,

SUM(1.0 / POWER(2, hops)) AS influence_score,

COUNT(DISTINCT src) AS reachers

FROM best GROUP BY dst ORDER BY influence_score DESC;

```

Answer 34

```

-- Diameter = max of shortest paths between all leaf pairs

WITH RECURSIVE tree AS (

SELECT cat_id, parent_id, 0 AS depth FROM category WHERE parent_id IS NULL

UNION ALL

SELECT c.cat_id, c.parent_id, t.depth + 1

FROM category c JOIN tree t ON c.parent_id = t.cat_id

),

leaves AS (SELECT cat_id FROM category c WHERE NOT EXISTS

(SELECT 1 FROM category ch WHERE ch.parent_id = c.cat_id)),

paths AS (

SELECT la.cat_id AS leaf_a, lb.cat_id AS leaf_b,

(SELECT MAX(depth) FROM category_closure cl

WHERE cl.ancestor IN (SELECT ancestor FROM category_closure

WHERE descendant = la.cat_id)

AND cl.ancestor IN (SELECT ancestor FROM category_closure

WHERE descendant = lb.cat_id)) AS lca_depth,

```

```

ta.depth + tb.depth AS sum_depth

FROM leaves la JOIN leaves lb ON la.cat_id < lb.cat_id

JOIN tree ta ON ta.cat_id = la.cat_id

JOIN tree tb ON tb.cat_id = lb.cat_id

)

SELECT MAX(sum_depth - 2 * lca_depth) AS tree_diameter FROM paths;

```

Answer 35

```

-- CHALLENGE: Full social graph report

WITH RECURSIVE closure AS (

SELECT follower_id AS src, followee_id AS dst, 1 AS hops FROM follows

UNION

SELECT c.src, f.followee_id, c.hops + 1

FROM closure c JOIN follows f ON f.follower_id = c.dst WHERE c.hops < 6

),

fwd AS (SELECT follower_id AS src, followee_id AS dst FROM follows

UNION SELECT c.src, f.followee_id FROM fwd c JOIN follows f ON f.follower_id = c.dst),

bwd AS (SELECT followee_id AS src, follower_id AS dst FROM follows

UNION SELECT b.src, f.follower_id FROM bwd b JOIN follows f ON f.followee_id = b.dst),

scc AS (SELECT f.src AS node_id, MIN(f.dst) AS scc_label

FROM fwd f JOIN bwd b ON f.src=b.dst AND f.dst=b.src GROUP BY f.src),

degrees AS (

SELECT follower_id AS node_id,

COUNT(*) AS out_deg FROM follows GROUP BY follower_id

),

in_deg AS (

SELECT followee_id AS node_id,

```

```
COUNT(*) AS in_deg FROM follows GROUP BY followee_id

),

sep AS (SELECT dst AS node_id, MIN(hops) AS sep_from_1

FROM closure WHERE src = 1 GROUP BY dst)

SELECT

u.user_id,

u.user_name,

COALESCE(id.in_deg, 0) AS in_degree,

COALESCE(d.out_deg, 0) AS out_degree,

COALESCE(scc.scc_label, u.user_id) AS scc_label,

COALESCE(sep.sep_from_1, -1) AS hops_from_user1

FROM users u

LEFT JOIN degrees d ON d.node_id = u.user_id

LEFT JOIN in_deg id ON id.node_id = u.user_id

LEFT JOIN scc ON scc.node_id = u.user_id

LEFT JOIN sep ON sep.node_id = u.user_id

ORDER BY hops_from_user1, in_degree DESC;
```

— End of Guide —

Hierarchies and graphs appear in every non-trivial database. Master the five storage models, recognise which pattern fits each problem, and your queries will be both correct and fast at any scale. That is the senior developer standard.